

Introduction to Structured Programming KU

Session 2: Pointer Arithmetic & Arrays

Sebastian David Albu

Summer Semester 2026

Institute of Software Engineering and Artificial Intelligence

LaTeX Template provided by the Institute of Information Security

Roadmap: Deep Dive



Arrays & Memory

Strings
vs. Pointers

Pointer Arithmetic

Double Pointers

Command Line
Arguments

Warning

This session covers the most common causes of **Segmentation Faults**. Pay attention to memory!

Recap: Advanced I/O

Format Specifiers

`printf` and `scanf` use placeholders to interpret raw bytes.

Specifier	Type
<code>%d</code>	Integer (Decimal)
<code>%x / %X</code>	Hexadecimal
<code>%f</code>	Float
<code>%c</code>	Character
<code>%s</code>	String
<code>%zu</code>	<code>size_t</code> (Sizes)
<code>%p</code>	Pointer Address

The ASCII Connection:

```
1 int num = 69;
2 printf("%d", num); // "69"
3 printf("%c", num); // "E"
4 printf("%x", num); // "45"
5
```

Data is just bits. The specifier tells C how to *interpret* it.

The ASCII Table: Everything is a Number

Computers only understand numbers. **ASCII** is the map we use to translate them into text.

Key mappings to remember:

- '0' → **48** (Not 0!)
- 'A' → **65**
- 'a' → **97**

The Difference

```
1 int a = 0;    // The value Zero
2 char b = '0'; // The symbol Zero (48)
3
```

Memory **Screen**

65	→	'A'
66	→	'B'

...

97	→	'a'
98	→	'b'

ASCII Arithmetic

Since char is just an integer, we can do math with it!

1. Convert Digit to Integer

```
1 char c = '5'; // ASCII 53
2 int val = c - '0';
3
4 // 53 - 48 = 5
5
```



2. Uppercase to Lowercase

```
1 char c = 'A'; // 65
2 c = c + 32; // 97 ('a')
3
```

By subtracting the base '0',
we get the real number!

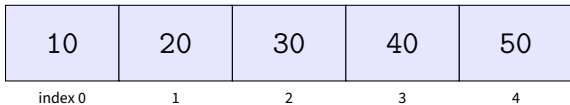
Arrays and Memory

Arrays in Memory

An array is a **contiguous** block of memory.

```
1 int nums[5] = {10, 20, 30, 40, 50};  
2
```

Total Size: $5 \times 4 \text{ bytes} = 20 \text{ bytes}$



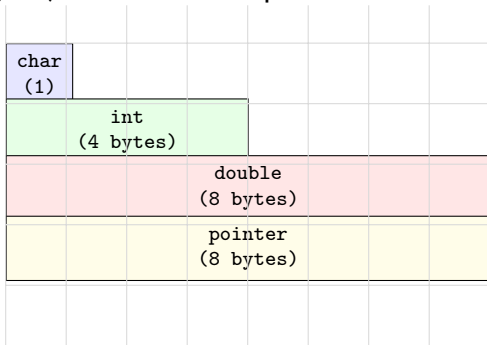
What is sizeof?

sizeof tells us how much memory (in bytes) a variable occupies.

Common Sizes:*

- char: **1 byte** (Always)
- int: **4 bytes** (Usually)
- double: **8 bytes**
- pointers: **Always 8 bytes**

*On modern 64-bit systems (LP64).

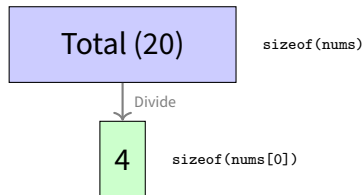


```
1 printf("%zu", sizeof(char)); // Prints 1
2 printf("%zu", sizeof(int));  // Prints 4
3
```

The sizeof Idiom

How do we calculate array length automatically?

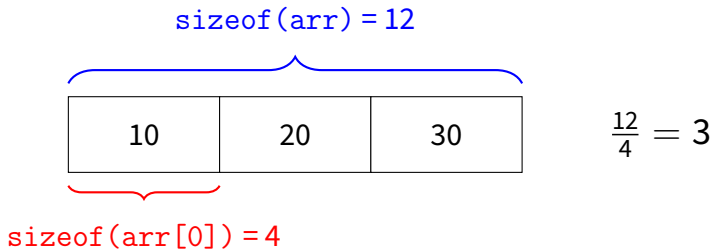
```
1 int nums[5] = {1, 2, 3, 4, 5};  
2  
3 // 1. Total bytes (20)  
4 size_t total = sizeof(nums);  
5  
6 // 2. Element bytes (4)  
7 size_t one = sizeof(nums[0]);  
8  
9 // 3. Count (20 / 4 = 5)  
10 size_t count = total / one;  
11
```



Calculating Array Length

Since `sizeof` returns **bytes**, not the element count, we must do math.

```
1 int arr[] = {10, 20, 30};  
2 // sizeof(arr)      -> 12 bytes (Total)  
3 // sizeof(arr[0])   ->  4 bytes (One Element)  
4
```



Strings

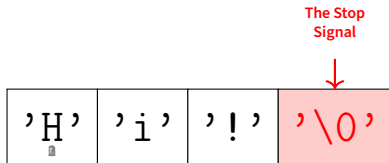
Strings: The "Fake" Data Type

C does not have a real string type. It just uses **Arrays of Characters**.
The Secret: A string is a sequence of characters ending with a special **Null Terminator** (`\0`).

```
1 // This...
2 char s[] = "Hi!";
3 // ...is actually this:
4 char s[] = {'H', 'i', '!', '\0'};
5
```

The Golden Rule

If you forget the `\0`, the computer doesn't know where the text ends and will print garbage memory!

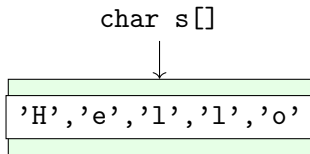


Strings: The Memory Trap

1. Mutable (Stack)

```
1 char s[] = "Hello";  
2 s[0] = 'B'; // OK!  
3
```

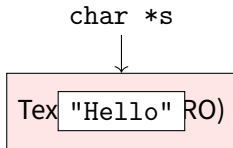
✓ **Works.** `s` is an array **copy** on the stack.



2. Read-Only (Literal)

```
1 char *s = "Hello";  
2 s[0] = 'B'; // CRASH!  
3
```

💣 **Segfault.** `s` points to **Read-Only Memory**.

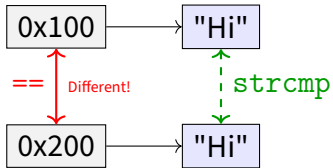


🚫 **Cannot Modify!**

Comparing Strings

Why `str1 == str2` fails.

```
1 char s1[] = "Hi";
2 char s2[] = "Hi";
3
4 if (s1 == s2) { ... }
5 // False! Checks Addresses!
6
7 if (strcmp(s1, s2) == 0) { ... }
8 // True! Checks Content.
9
```



Comparing Strings - Without string.h



```
1 int my_strncmp(const char *str1, const char *str2)
2 {
3     while (*str1 != '\0' && *str2 != '\0')
4     {
5         if (*str1 != *str2)
6         {
7             return *str1 - *str2;
8             // Return the difference between the characters
9         }
10        str1++; // Increase pointer -> move to next char
11        str2++;
12    }
13    return *str1 - *str2; // Will be 0 if equal
14 }
```

What is this magic??



Don't Worry

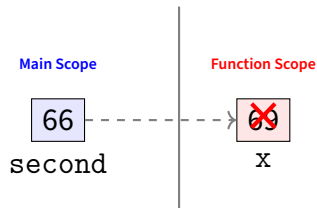
Pointers are your friends! :)

Pointers

Attempt 1: Pass by Value (The Copy)

In C, variables are passed by **value** (copied) by default.

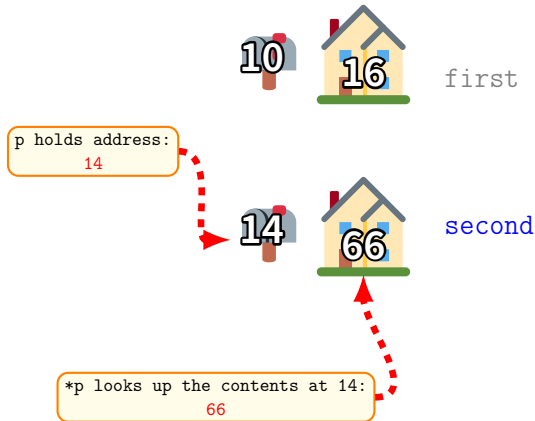
```
1 void add3(int x) {  
2     // x is a local COPY  
3     x = x + 3;  
4     // x becomes 69 here...  
5     // ...but is destroyed  
6     // when function ends!  
7 }  
8 int main(void) {  
9     int first = 16;  
10    int second = 66;  
11    add3(second);  
12    // 'second' is STILL 66!  
13 }
```



Pass by Reference

Every variable value is stored at an **address** in memory. We can pass this address.

```
1 void add3(int *p) {  
2     // '*p' -> Lookup contents  
3     *p = *p + 3;  
4 }  
5 int main() {  
6     int first = 16;  
7     int second = 66;  
8     // Pass the 'mail' address  
9     add3(&second);  
10    // 'second' is now 69  
11 }
```



Pointers Cheatsheet

- `int local_variable;`
→ Creates a variable which stores an integer **value**.
- `int *addr;`
→ Creates a variable which stores an **address** pointing to an integer.
- `addr = &local_variable;`
→ Sets the value of `addr` to the address of `local_variable`.
- `*addr = 5;`
→ Writes to the address location.
- **Result:** `local_variable` will now have the value 5.

Passing Arrays in Functions

Arrays are always passed by reference in C

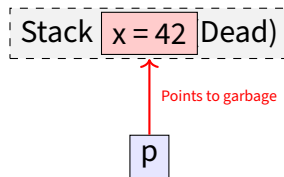
Be careful with how you pass arrays to functions.

The Zombie Pointer (Dangling)

Termination Code 139 (Segfault)

Never return the address of a **local** variable.

```
1 int* getNum()  
2 {  
3     int x = 42;  
4     return &x;  
5     // x dies here!  
6 }  
7  
8 int main()  
9 {  
10    int* p = getNum();  
11    printf("%d", *p);  
12    // BOOM!  
13 }  
14
```



Pointer Arithmetics

The Truth About Array Indexing

To the C compiler, square brackets `[]` do not exist. They are just **Syntactic Sugar**.
What you write:

```
1 x = arr[2];
```

2

What the Compiler sees:

```
1 x = *(arr + 2);
```

2

The Golden Rule:

$E1[E2] \equiv *(E1 + E2)$

The Truth About Array Indexing



Explanation

1. Take the address `arr` (start of array).
2. Move forward by 2 steps (elements).
3. `*` (Dereference): Go get the value at that new address.

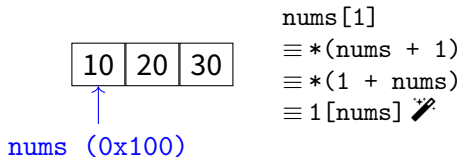
Arrays are easy, right?

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int numbers[3] = {10, 20, 30};
6     int i = 2;
7     printf("%d\n", numbers[i]); // OK This makes sense
8     printf("%d\n", *(numbers + i)); // Hmm yeah why not?
9     printf("%d\n", *(&numbers[i])); // Unneccessary?
10    printf("%d\n", i[numbers]); // ???
11    return 0;
12 }
```

Pointer Arithmetic: The `0[i]` Magic

`arr[i]` is syntactic sugar for `*(arr + i)`.

```
1 int nums[] = {10, 20, 30};  
2 int *p = nums;  
3
```



Why `i[arr]` Works (The Math)

Since indexing is just addition, the **Commutative Property** applies ($A + B = B + A$).

`arr[i] → *(arr + i)`

`i[arr] → *(i + arr)`

Since `arr + i` is the same as `i + arr`:

`arr[i]` is exactly the same as `i[arr]`

Please don't actually
write `i[arr]` in
production code!



Why `i[arr]` Works (The Math)

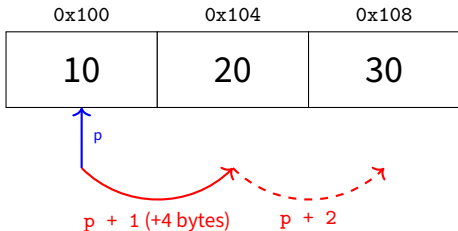


```
1 int nums[] = {10, 20, 30};  
2 int x = 2[nums]; // Valid C! Becomes *(2 + nums)  
3 // x is 30  
4
```

Pointer Arithmetic: The Scale Factor

Warning: Adding +1 to a pointer does **NOT** add 1 byte. It adds **1 Element Size** (e.g., 4 bytes for int).

```
1 int *p = 0x100; // Let's say p is at address 100
2 p = p + 1;      // p is now 0x104 (NOT 0x101!)
3
```



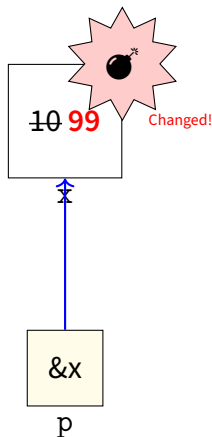
Dereferencing: Action at a Distance

Using `*p` acts like a remote control for `x`.

```
1 int x = 10;  
2 int *p = &x;  
3  
4 // The Magic Line  
5 *p = 99;  
6
```

Read it as:

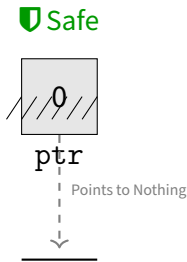
"Follow the pointer `p` to its destination, and overwrite THAT memory with 99."



The Safe Pointer: NULL

A pointer that points to "nowhere" is dangerous. Always initialize unused pointers to NULL.

```
1 // 1. Safe Initialization
2 int *p = NULL;
3 ...
4 // 2. Check before use!
5 if (p != NULL) {
6     printf("%d", *p);
7 } else {
8     printf("Pointer is empty!");
9 }
```



The Safe Pointer: NULL



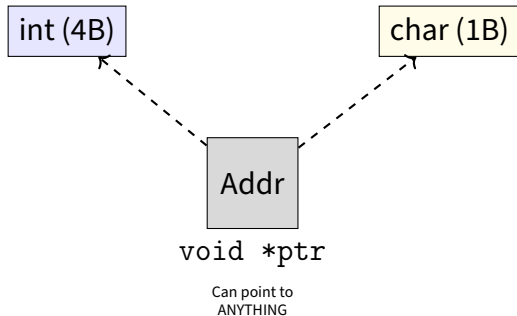
Warning

Dereferencing NULL (`*p` when `p` is NULL) crashes the program immediately. This is better than corrupting random memory!

The Void Pointer (void*)

A void* is a **generic pointer**. It holds an address, but has no type information.

```
1 int i = 10;
2 char c = 'A';
3
4 // The "Wildcard"
5 void *ptr;
6
7 ptr = &i; // OK! Points to int
8 ptr = &c; // OK! Points to char
9
```



Think of it as a box with a blank label. You know WHERE it is, but not WHAT is inside.

The Trap: Dereferencing Void

The Rule

You **cannot** dereference a `void*` directly. The compiler doesn't know how many bytes to read!

The Error:

```
1 void *p = &x;  
2  
3 // COMPILER ERROR!  
4 // How big is *p? 1 byte? 4?  
5 printf("%d", *p);  
6
```

The Fix (Type Casting):

```
1 // "Treat this address as an  
   int*"   
2 printf("%d", *(int*)p);  
3
```

Quick Detour: Type Casting

Type Casting: Implicit vs. Explicit

Casting is the conversion of one data type into another.

1. Implicit (Automatic)

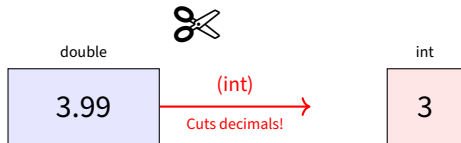
```
1 int i = 5;  
2 // "Widening" -> Safe!  
3 double d = i;  
4 // d becomes 5.0  
5
```

Compiler handles this. No data loss.

2. Explicit (Manual)

```
1 double pi = 3.99;  
2 // "Narrowing" -> Loss!  
3 int i = (int)pi;  
4 // i becomes 3 (Truncated)  
5
```

You force this using (type).



The Trap: Integer Division

Rule of Math

Integer / Integer = Integer (Decimals are lost!)

❌ The Mistake

```
1 int sum = 5;
2 int count = 2;
3
4 // 5 / 2 = 2 (Truncated)
5 // 2 -> 2.0
6 double avg = sum / count;
7
8 printf("%.1f", avg);
9 // Prints "2.0"
```

✅ The Fix (Cast)

```
1 int sum = 5;
2 int count = 2;
3
4 // Promote ONE to double
5 // 5.0 / 2 = 2.5
6 double avg = (double)sum /
7               count;
8
9 printf("%.1f", avg);
10 // Prints "2.5"
```

The Trap: Integer Division

Note: `(double)(sum / count)` is **WRONG!**
It divides first (2), then casts to (2.0). Cast **before** dividing!

Self Check

Comparing Strings - Without string.h



```
1 int my_strncmp(const char *str1, const char *str2)
2 {
3     while (*str1 != '\0' && *str2 != '\0')
4     {
5         if (*str1 != *str2)
6         {
7             return *str1 - *str2;
8             // Return the difference between the characters
9         }
10        str1++; // Increase pointer -> move to next char
11        str2++;
12    }
13    return *str1 - *str2; // Will be 0 if equal
14 }
```

Does this now make more sense?



Don't Worry

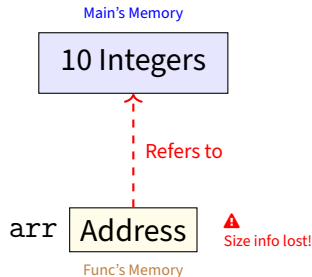
Pointers are hard at first - Practice with simple functions like custom strcmp, strlen, ...

Arrays as Function Arguments

The Secret

You **cannot** pass a full array to a function. C always passes a **Pointer** to the first element.

```
1 // These are IDENTICAL:
2 // void func(int arr[])
3 void func(int *arr) {
4     arr[0] = 99; // Changes Original!
5     // WARNING:
6     // sizeof(arr) is now 8 (ptr size)
7     // NOT the array size!
8 }
9
10 int main() {
11     int data[10];
12     func(data); // Passes &data[0]
13 }
14
```

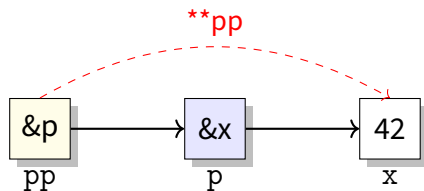


Double Pointers

Double Pointers (**)

A pointer stores an address. A double pointer stores the address of a pointer.

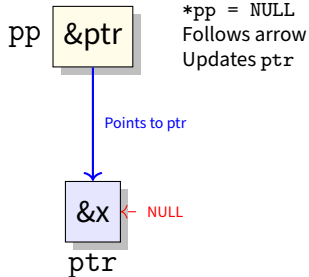
```
1 int x = 42;  
2 int *p = &x;  
3 int **pp = &p;  
4  
5 // **pp -> *p -> x  
6
```



Why use **pp? (Modifying Pointers)

If you want a function to change *where a pointer points*, you need a double pointer.

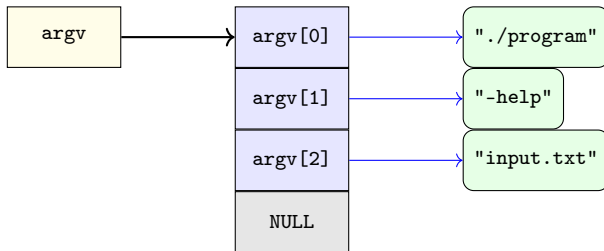
```
1 void set_to_null(int **pp) {  
2     // *pp is the original pointer  
3     *pp = NULL;  
4 }  
5  
6 int main() {  
7     int x = 5;  
8     int *ptr = &x;  
9  
10    // We want to modify 'ptr' itself  
11    set_to_null(&ptr);  
12  
13    // ptr is now NULL  
14    if (ptr == NULL) printf("Done");  
15 }  
16
```



Real World: char **argv

A common use of double pointers is an **Array of Strings** (like command line arguments).

```
1 int main(int argc, char **argv) { ... }  
2
```

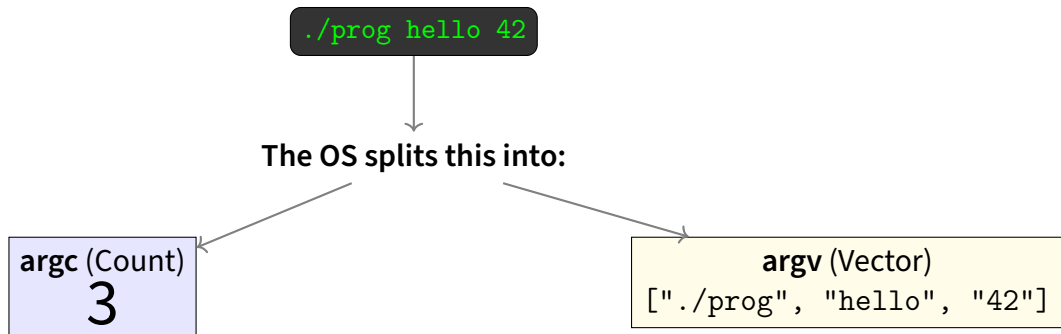


argv points to an array... which contains pointers... which point to strings!

Coming Up: Talking to `main()`

How do we send data to the program **before** it even starts?

```
./minecraft -username Steve -server 127.0.0.1
```



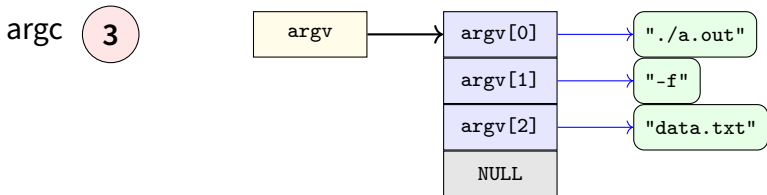
Next Session: Command Line Arguments & parsing strings!

Command Line Arguments

Understanding argc and argv

`./program arg1 arg2`

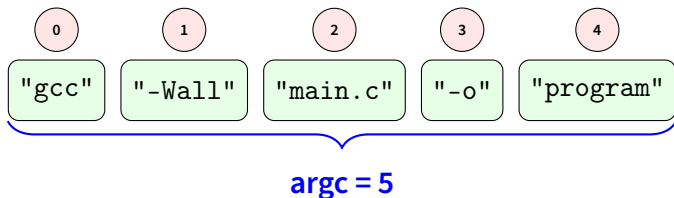
- **argc (Argument Count):** How many items? (Includes program name!)
- **argv (Argument Vector):** An array of strings.



How Arguments are Created

When you type a command, the shell **tokenizes** it (splits by spaces).

```
gcc -Wall main.c -o program
```



Note: `argv[0]` is the command itself (`"gcc"`). The first "real" user argument is `argv[1]`.

Iterating Through Arguments

To print all arguments, we loop from 0 to `argc - 1`.

```
1 int main(int argc, char **argv)
2 {
3     // Safe Loop
4     for (int i = 0; i < argc; i++)
5     {
6         printf("Arg %d: %s\n",
7               i, argv[i]);
8     }
9
10    return 0;
11 }
12
```

Safety Check

Never access `argv[1]` without checking if `argc >= 2` first!

Parsing Numbers: atoi

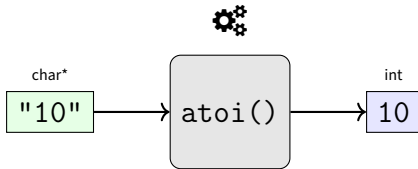


argv always contains **Strings**. Even if you type "42", it is stored as '4', '2', '\0'.
Problem:

```
1 // User runs: ./prog 10
2 int x = argv[1]; // WRONG! (Type Mismatch)
3
```

Solution:

```
1 #include <stdlib.h> // for atoi
2
3 // "ASCII to Integer"
4 int x = atoi(argv[1]);
5 int res = x + 5; // Now 15!
6
```



No Arguments Needed? (void)

If your program does not accept command line arguments, make it explicit!

? Implicit (Okay)

```
1 int main()  
2 {  
3     // Accepted in C,  
4     // DEPENDING on used  
5     compiler  
6     return 0;  
7 }
```

✓ Explicit (Best)

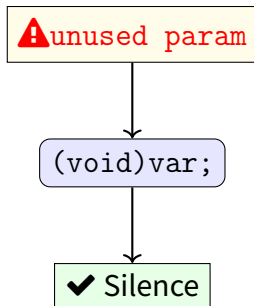
```
1 int main(void)  
2 {  
3     // Explicitly states:  
4     // "I take NO arguments."  
5     return 0;  
6 }  
7
```

Note: `main()` works with some compilers, but `main(void)` is stricter and safer.

Suppressing "Unused Variable" Warnings

Sometimes you **must** accept arguments (to match a signature), but you don't use them. The compiler will warn you!

```
1 int main(int argc, char **argv)
2 {
3     // "Touch" the variable to silence
4     // the compiler warning.
5     (void) argc;
6     (void) argv;
7     printf("I ignore my inputs!\n");
8     return 0;
9 }
```



This tells the compiler: *"I know this variable exists, and I am intentionally ignoring it."*

Questions?
(Before we Segfault?)

Introduction to Structured Programming KU

Session 2: Pointer Arithmetic & Arrays

Sebastian David Albu

Summer Semester 2026

Institute of Software Engineering and Artificial Intelligence

LaTeX Template provided by the Institute of Information Security